Izpitni povzetek A4 v2

Izpitni povzetek A4 - zgoščena popravljena verzija

Asimptotika

O zgornja, Ω spodnja, Θ tesna. Ignoriraj konstante/nižje člene. `1+...+n=Θ(n^2)`. Poln graf: `m=Θ(n^2)`. 1 GHz ≈ `10^9` op/s. `log n < n < n log n < n^2 < n^3 < 2^n < n!`.

Rekurzije: `T(n)=T(n/2)+1 -> log n`; `2T(n/2)+n -> n log n`; `T(n-1)+1 -> n`; Fibonacci brez memo -> eksponentno.

Tabele/ADT

ADT = operacije + pomen; PS = implementacija. Tabela `get/put O(1)`. Dinamična tabela s podvajanjem: `insert O(1)` amortizirano, ker kopiranja `1+2+4+...<2n`.

Tabela tabel: `A_i` velikosti `2^i`; zasedene tabele = biti v binarnem zapisu n; iskanje `O(log^2 n)`; vstavljanje kot binarni števec z zlivanjem.

Seznami

Povezan seznam: iskanje `O(n)`. Urejenost sama ne pomaga pri kazalčnem seznamu. Preskočni seznam: nivo z meti kovanca; search/insert pričakovano `O(log n)`, prostor `O(n)`.

Zgoščevanje

`h(k)=k mod m` ali `(k*p) mod m`. Faktor zasedenosti $\alpha=n/m$. Veriženje: $O(1+\alpha)$. Linearno: `h(k,i)=(h(k)+i) mod m`; brisanje z `DELETED`; pazi na gruče in polno tabelo. Slovar: isti ključ navadno posodobi vrednost.

Bloom: vstavi = prižgi k bitov; query: vsaj en 0 -> zagotovo NE, vsi 1 -> verjetno DA. False positive da, false negative ne. Brisanje: counting BF.

Kopice / PQ

Min-kopica: starš <= otroci. `min O(1)`, `insert O(log n)`, `deleteMin O(log n)`, `heapify O(n)`. Kopica ni BST; max v min-kopici je med listi.

Drevesa

BST: levo manjše, desno večje. `search/insert/delete O(h)`. Min = skrajno levo, max = skrajno desno.

AVL: `|hL-hR|<=1`; LL desna, RR leva, LR leva+desna, RL desna+leva. Vse `O(log n)`.

RČ: koren črn; rdeč nima rdečega otroka; vse poti imajo isto črno višino; novo vozlišče rdeče; rdeč stric -> prebarvanje, črn stric -> rotacija. AVL bolj iskalno strog, RČ manj rotacij.

B/B+: več ključev v vozlišču, vsi listi ista globina. B+ ima podatke v listih, notranja vozlišča indeks.

Disjunktne množice

`makeSet/find/union`; gozd s parent kazalci; path compression + union by rank/size -> skoraj `O(1)`. Kruskal za cikle.

Amortizacija

Vsota: skupno delo / št. operacij. Dvojiški števec: `n+n/2+...<2n -> O(1)`. Sklad/multipop: element enkrat noter, enkrat ven -> `O(1)`. Potencial: `a_i=c_i+Phi(D_i)-Phi(D_{i-1})`.

DP

Vedno: stanje, prehod, osnove, vrstni red, rekonstrukcija.

Stopnice 1/2: `f(n)=f(n-1)+f(n-2)`, `f(0)=1`, `f(1)=1`.

0/1 nahrbtnik: `dp[i][w]=max(dp[i-1][w], dp[i-1][w-w_i]+v_i)` če `w_i<=w`; drugače `dp[i-1][w]`. Čas `O(nW)`.

Delni nahrbtnik: sort po `v/w`, jemlji greedy.

Matrix-chain: `M_i=d_{i-1} x d_i`; `c(i,j)=min_k(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)`, `c(i,i)=0`, po diagonalah, čas `O(n^3)`.

Grafi

Matrika: prostor `O(n^2)`, edge check `O(1)`, sosedi `O(n)`. Seznam: prostor `O(n+m)`, sosedi hitro.

MST: `n-1` povezav, brez cikla, povezan. Različne uteži -> enoličen MST.

Prim: širi drevo z najcenejšo povezavo iz `T` v `V-T`; `O(n^2)` matrika, `O((n+m)log n)` heap.

Kruskal: sort edge po teži, dodaj če DSU-find različna; `O(m log m)`.

Topološko: samo DAG; indegree 0 v vrsto; odstranjuj; če ostane cikel. DAG shortest path: topo + relax `O(n+m)`.

Dijkstra: nenegativne uteži; vedno izberi neobiskano min `d`; relax. Negativna utež -> NE.

Bellman-Ford: negativne uteži OK; `n-1` krat relax vse edge; dodatni prehod zazna negativen cikel; `O(nm)`.

Welsh-Powell: vozlišča padajoče po stopnji; nova barva za prvo nepobarvano; z isto barvo barvaj nesosednja. Ni nujno optimalno.

Največji pretok: preskoči.

P/NP

P: rešljivo polinomsko. NP: rešitev preverljiva polinomsko. $P \subseteq NP$. Redukcija `A <=p B` pomeni B vsaj tako težak kot A. NP-poln = v NP + vsi NP nanj. NP-težak = vsaj tako težak, ni nujno v NP. Algoritem ≠ problem.